



第1章习题

1.1 简答题

- (1) 哪个处理器的指令系统称为 80x86 系列处理器的基本指令集?
- (2) ROM-BIOS 是什么?
- (3) 什么是通用寄存器?
- (4) 堆栈的存取原则是什么?
- (5) 标志寄存器主要保存哪方面的信息?
- (6) 最高有效位 (MSB) 是指哪一位?
- (7) 汇编语言中的标识符与高级语言的变量和常量名的组成原则有本质的区别吗?
- (8) 汇编语言的标识符大小写不敏感意味着什么?

- (9) 在汇编语言源程序文件中, END 语句后的语句会被汇编吗?
- (10) 汇编时生成的列表文件主要包括哪些内容?

1.2 判断题

- (1) EAX 也被称为累加器,因为它使用最频繁。
- (2) 指令指针寄存器 EIP 属于通用寄存器。
- (3) IA-32 处理器在实地址方式下,不能使用 32 位寄存器。
- (4) 保护方式下,段基址加偏移地址就是线性地址或物理地址。
- (5) Windows 的模拟 DOS 环境与控制台环境是一样的。
- (6) 处理器的传送指令 MOV 属于汇编语言的执行性语句。
- (7) 汇编语言的语句由明显的 4 部分组成,不需要用分隔符区别。
- (8) MASM 汇编语言的注释以分号开始,但不能用中文分号。
- (9) 程序终止执行也就意味着汇编结束,所以两者含义相同。
- (10) 源程序文件和列表文件都是文本性质的文件。

1.3 填空题

- (1) Intel 8086 支持_____容量主存空间, IA-32 处理器支持_____容量主存空间。
- (2) Intel_____处理器将 80x86 指令系统升级为 32 位指令系统,_____处理器内部集成浮点处理单元,支持浮点操作指令。
- (3) IA-32 处理器有 8 个 32 位通用寄存器,其中 EAX、_____,_____和 EDX 可以分成 16 位和 8 位操作,另外 4 个是_____,_____,_____和_____。
- (4) 寄存器 EDI 是_____位的,其中低 16 位的名称是_____,它还可以分成两个 8 位的寄存器,其中 $D_0 \sim D_7$ 和 $D_8 \sim D_{15}$ 部分可以分别用_____和_____表示。
- (5) IA-32 处理器有_____个段寄存器,它们都是_____位的。
- (6) IA-32 处理器复位后,首先进入的是_____工作方式。该工作方式的分段最大不超过_____。
- (7) 逻辑地址由_____和_____两部分组成。代码段下一条要执行的指令由 CS 和_____寄存器指示,后者在实地址模型中起作用的仅有_____寄存器部分。
- (8) Windows 的文件夹对应的专业术语是_____。
- (9) 指令由表示指令功能的_____和表示操作对象的_____部分组成, IA-32 处理器的指令前缀属于_____部分。
- (10) MASM 要求汇编语言源程序文件的扩展名是_____,汇编产生的扩展名为 OBJ 的文件称为_____文件,编写 32 位 Windows 应用程序应选择_____存储模型。
- 1.4 简述 Intel 80x86 系列处理器在指令集方面的发展。
- 1.5 说明微型计算机系统的硬件组成及各部分的作用。
- 1.6 什么是标志?什么是 IA-32 处理器的状态标志、控制标志和系统标志?说明状态标志在标志寄存器 EFLAGS 中的位置和含义。
- 1.7 什么是平展存储模型、段式存储模型和实地址存储模型?
- 1.8 什么是实地址方式、保护方式和虚拟 8086 方式?它们分别使用什么存储模型?
- 1.9 IA-32 处理器有哪三类基本段,各有什么用途?
- 1.10 说明高级语言、汇编语言、机器语言三者的区别,谈谈你对汇编语言的认识。
- 1.11 区别如下概念:助记符、汇编语言、汇编语言程序和汇编程序。
- 1.12 区别如下概念:路径、绝对路径、相对路径、当前目录。系统磁盘上存在某个可执行文件,但在 DOS 环境下输入其文件名却提示没有这个文件,原因是什么?
- 1.13 汇编语言语句有哪两种,每种语句由哪 4 部分组成?
- 1.14 什么是标识符和保留字,汇编语言程序中的标识符是怎样组成的?
- 1.15 在 MASM 汇编语言中,下面哪些是程序员可以使用的自定义标识符?
FFH, DS, Again, next, @data, h_ascii, 6364b, flat
- 1.16 汇编语言程序的开发有哪 4 个步骤,分别利用什么程序完成、产生什么输出文件?

第2章习题

2.1 简答题

- (1) 使用8位二进制表达无符号整数, 257 有对应的编码吗?
- (2) 字符 'F' 和数值 46H 作为 MOV 指令的源操作数有区别吗?
- (3) 为什么可以把指令 “MOV EAX, (34 + 67H) * 3” 中的数值表达式看成是常量?
- (4) 汇编语言为什么规定十六进制数若以 A~F 开头, 需要在前面加个 0?
- (5) 数值 500 能够作为字节变量的初值吗?
- (6) 多字节数据对齐地址边界有什么作用?
- (7) 为什么将查找操作数的方法称为数据寻 “址” 方式?
- (8) 为什么变量 VAR 在指令 “MOV EAX, VAR” 中表达直接寻址?

32 位汇编语言程序设计 第3版

- (9) 指令 “MOV EAX, [ESI]” 从哪个段获得存储器操作数?
- (10) 为什么带比例的变址寻址中的比例只设计支持 1、2、4 和 8?

2.2 判断题

- (1) 对一个正整数, 它的原码、反码和补码都一样, 也都与无符号数的编码一样。
- (2) 常用的 BCD 码为 8421 BCD 码, 其中的 8 表示 D_3 位的权重。
- (3) 排序一般按照 ASCII 码值大小, 从小到大升序排列时, 小写字母排在写字母之前。
- (4) 用 DB 和 DD 定义变量, 如果初值相同, 则占用的存储空间也一样多。
- (5) TYPE DX 的结果是一个常量, 等于 2。
- (6) IA-32 处理器采用小端方式存储多字节数据。
- (7) 某个双字变量存放于存储器地址 0403H ~ 0406H 中, 对齐了地址边界。
- (8) 立即数寻址只会出现在源操作数中。
- (9) 存储器寻址方式的操作数当然在主存了。
- (10) 指令 “MOV EAX, [VAR + 2]” 与 “MOV EAX, VAR [2]” 功能相同。

2.3 填空题

- (1) 计算机中有一个 “01100001” 编码。如果认为它是无符号数, 它是十进制数 _____; 如果认为它是 BCD 码, 则表示真值 _____; 如果认为它是某个 ASCII 码, 则代表字符 _____。
- (2) C 语言用 “\n” 表示让光标回到下一行首位, 在汇编语言中需要输出两个控制字符: 一个是回车, 其 ASCII 码是 _____, 它将光标移动到当前所在行的首位; 另一个是换行, 其 ASCII 码是 _____, 它将光标移到下一行。
- (3) 定义字节变量的伪指令助记符是 _____, 获取变量名所具有的偏移地址的操作符是 _____。
- (4) 数据段有语句 “H8843 DD 99008843H”, 代码段指令 “MOV CX, WORD PTR [H8843]” 执行后, CX = _____。
- (5) 用 DD 定义的一个变量 XYZ, 它的类型是 _____, 用 “TYPE XYZ” 会得到数值为 _____。如果将其以字量使用, 应该用 _____ 说明。
- (6) 数据段有语句 “ABC DB 1, 2, 3”, 代码段指令 “MOV CL, [ABC + 2]” 执行后, CL = _____。
- (7) 除假设数据外的数据寻址方式有 3 类, 分别称为 _____、_____ 和 _____。
- (8) 指令 “MOV EAX, OFFSET MSG” 的目的操作数和源操作数分别采用 _____ 和 _____ 寻址方式。
- (9) 已知 ESI = 04000H, EBX = 20H, 指令 “MOV EAX, [ESI + EBX * 2 + 8]” 中访问的有效地址是 _____。
- (10) 用 EBX 作为基址指针, 默认采用 _____ 段寄存器指向的数据段; 如果采用 BP、EBP 或 SP、ESP 作为基址指针, 默认使用 _____ 段寄存器指向堆栈段。

2.4 下列十六进制数表示无符号整数, 请转换为十进制形式的真值:

- (1) FFH
- (2) 0H
- (3) 5EH
- (4) EFH

2.5 将下列十进制数真值转换为压缩 BCD 码:

- (1) 12
- (2) 24
- (3) 68
- (4) 99

2.6 将下列压缩 BCD 码转换为十进制数:

- (1) 10010001
- (2) 10001001
- (3) 00110110
- (4) 10010000

2.7 将下列十进制数用 8 位二进制补码表示:

- 2.8 进行十六进制数据的加减运算，并说明是否有进位或借位：
 (1) $1234H + 7802H$ (2) $F034H + 5AB0H$
 (3) $C051H - 1234H$ (4) $9876H - ABCDH$
- 2.9 数 0~9、大写字母 A~Z、小写字母 a~z 对应的 ASCII 码分别是多少？ASCII 码 0DH 和 0AH 分别对应什么字符？
- 2.10 设置一个数据段，按照如下要求定义变量或符号常量：
 (1) my1b 为字符串变量：Personal Computer
 (2) my2b 为用十进制数表示的字节变量：20
 (3) my3b 为用十六进制数表示的字节变量：20
 (4) my4b 为用二进制数表示的字节变量：20
 (5) my5w 为 20 个未赋值的字变量
 (6) my6c 为 100 的常量
 (7) my7e 表示字符串：Personal Computer

- 2.11 定义常量 NUM，其值为 5；数据段中定义字数组变量 DATA1ST，它的头 5 个字节单元中依次存放 -10、2、5 和 4，最后 1 个字节单元初值不定。

- 2.12 从低地址开始以字节为单位，用十六进制形式给出下列语句依次分配的数值：

```
db 'ABC',10,10h,'EF',3 dup(-1,2,3 dup(4))
dw 10h,-5,3 dup(0)
```

- 2.13 设在某个程序中有如下片段，请写出每条传送指令执行后寄存器 EAX 的内容：

```
; 数据段
varw dw 1234h,5678h
varb db 3,4
count1 equ $ - varw
vard dd 12345678h
count2 equ $ - vard
buff db 10 dup(0)
mess db 'hello'
count3 equ $ - buff
; 代码段
mov eax,offset mess
mov eax,count1
mov eax,count2
mov eax,count3
```

- 2.14 按照如下输出格式，在屏幕上显示 ASCII 表：

```
  | 0 1 2 3 4 5 6 7 8 9 A B C D E F
  +-----+
20 | ! " # $ % & ' ( ) * + , - . /
30 | 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
40 | @ A B C D E F G H I J K L M N O
50 | P Q R S T U V W X Y Z [ \ ] ^ _
60 | ` a b c d e f g h i j k l m n o
70 | p q r s t u v w x y z { | } ~
```

表格最上一行的数字是对应 ASCII 代码值的低 4 位（用十六进制形式），而表格左边的数字对应 ASCII 代码值的高 4 位（用十六进制形式）。编程在数据段直接构造这样的表格，填写相应 ASCII 代码值（不是字符本身），然后实现显示。

86 32 位汇编语言程序设计 第 3 版

- 2.15 数据段有如下定义，IA-32 处理器将以小端方式保存在主存：

```
var dd 12345678h
```

以字节为单位按地址从低到高的顺序，写出这个变量内容，并说明如下指令的执行结果：

```
mov eax,[var]          ;EAX=_____
mov bx,word ptr [var]   ;BX=_____
mov cx,word ptr [var+2] ;CX=_____
mov dl,byte ptr [var]   ;DL=_____
mov dh,byte ptr [var+3] ;DH=_____
```

可以编程使用十六进制字节显示子程序 DSIPHB 顺序显示各个字节进行验证，还可以使用十六进制双字显示子程序 DSIPHD 显示该数据进行对比。

- 2.16 给出 IA-32 处理器 32 位寻址方式的组成公式，并说明各部分的作用。

- 2.17 给出下列指令中源操作数的寻址方式。假设 VARD 是一个双字变量。

```
(1) mov edx,1234h
(2) mov edx,[dvar]
(3) mov edx,ebx
(4) mov edx,[ebx]
(5) mov edx,[ebx+1234h]
(6) mov edx,var[ebx]
(7) mov edx,[ebx+edi]
(8) mov edx,[ebx+edi+1234h]
(9) mov edx,var[esi+edi]
(10) mov edx,[ebp*4]
```

第3章习题

3.1 简答题

- (1) 如何修改“MOV ESI, WORD PTR 250”语句使其正确?
- (2) 为什么说“XCHG EDX, CX”是一条错误的指令?
- (3) 说IA-32处理器的堆栈段“向下生长”是什么意思?
- (4) 都是获取偏移地址,为什么指令“LEA EBX, [ESI]”正确,而指令“MOV EBX, OFFSET[ESI]”错误?
- (5) 执行了一条加法指令后,发现ZF=1,说明结果是什么?
- (6) INC, DEC, NEG和NOT都是单操作数指令,这个操作数应该是源操作数还是目的操作数?
- (7) 大小写字母的转换使用了什么规律?
- (8) 乘除法运算针对无符号数和有符号数,有两种不同的指令。只有一种指令的加减法如何区别无符号数和有符号数运算?
- (9) 除法指令“DIV ESI”的被除数是什么?
- (10) 逻辑与运算为什么也称为逻辑乘?

3.2 判断题

- (1) 指令“MOV EAX, 0”使EAX结果为0,所以标志ZF=1。
- (2) 空操作NOP指令其实根本没有指令。
- (3) 堆栈的操作原则是“先进后出”,所以堆栈段的数据除PUSH和POP指令外,不允许其他方法读写。
- (4) 虽然ADD指令和SUB指令执行后会影响标志状态,但执行前的标志并不影响它们的执行结果。
- (5) 80减90(80-90)需要借位,所以执行结束后,进位标志CF=1。
- (6) 指令“INC ECX”和“ADD ECX,1”实现的功能完全一样,可以互相替换。
- (7) 无符号数在前面加零扩展,数值不变;有符号数前面进行符号扩展,位数加长一位,数值增加一倍。
- (8) CMP指令是目的操作数减去源操作数,与SUB指令功能相同。

32位汇编语言程序设计 第3版

- (9) 逻辑运算没有进位或溢出问题,此时CF和OF没有作用,所以逻辑运算指令(如AND、OR等)将CF和OF设置为0。
- (10) SHL指令左移一位,就是乘10。

3.3 填空题

- (1) 指令“PUSH DS”执行后,ESP会_____。
- (2) 指令“POP EDX”的功能也可以用MOV和ADD指令实现,依次应该是_____和_____指令。
- (3) 例3-3的TAB定义如果是“1234567890”,则显示结果是_____。
- (4) 进行8位二进制数加法:BAH+6CH,8位结果是_____,标志PF=_____。如果进行16位二进制数加法:45BAH+786CH,16位结果是_____,标志PF=_____。
- (5) 已知AX=98H,执行“NEG AX”指令后,AX=_____,标志SF=_____。
- (6) 假设CL=98H,执行“MOVZX DX,CL”指令后,DX=_____,这称为_____扩展。
- (7) 假设CL=98H,执行“MOVZX DX,CL”指令后,DX=_____,这称为_____扩展。
- (8) 执行指令“XOR EAX,EAX”和“SUB EAX,EAX”后,EAX=_____,CF=OF=_____。而执行指令“MOV EAX,0”后,EAX=_____,CF和OF没有变化。
- (9) 例3-9的程序执行结束后,变量QVAR的内容是_____。
- (10) 欲将EDX内的无符号数除以16,使用指令“SHR EDX,_____”,其中后一个操作数是一个立即数。

3.4 MOV指令支持多种操作数组合,请给每种组合举一个实例。

- (1) mov reg,imm
- (2) mov mem,imm
- (3) mov reg,reg
- (4) mov mem,reg
- (5) mov seg,reg
- (6) mov reg,mem
- (7) mov seg,mem
- (8) mov reg,seg
- (9) mov mem,seg

3.5 操作数的组合通常符合逻辑,但不是任意的,指出如下指令的错误原因。

- (1) mov ecx,dl
- (2) mov eip,ax
- (3) mov es,1234h
- (4) mov es,ds
- (5) mov al,300
- (6) mov [esi],45h
- (7) mov eax,ebx+edi
- (8) mov 20h,ah

3.6 使用MOV指令实现交换指令“XCHG EBX,[EDI]”的功能。

3.7 什么是堆栈?它的工作原理是什么?它的基本操作有哪两个?对应哪两种指令?

- 3.8 假设当前 ESP=0012F00H, 说明下面每条指令后 ESP 等于多少。

```
push eax
push dx
push dword ptr 0779h
pop eax
pop word ptr [bx]
pop esi
```

- 3.9 已知数字 0~9 对应的寄存器依次为 18H, 34H, 05H, 06H, 09H, 0AH, 0CH, 11H, 12H, 14H, 请为如下程序的每条指令加上注释, 说明每条指令的功能和执行结果。

```
; 数据段
table db 18h, 34h, 05h, 06h, 09h, 0ah, 0ch, 11h, 12h, 14h
; 代码段
mov ebx, offset table
mov al, 0
xlat
```

为了验证你的判断, 不妨使用本书的 I/O 子程序库提供的 L/O DISPHB 显示码后 AL 的值。如果不使用 XLAT 指令, 应如何修改?

- 3.10 举例说明 CF 和 OF 标志的差异。

- 3.11 分别执行如下程序片段, 说明每条指令的执行结果:

```
(1) mov eax, 80h ; EAX = _____
    add eax, 3 ; EAX = _____, CF = _____, OF = _____
    add eax, 80h ; EAX = _____, CF = _____, OF = _____
    adc eax, 3 ; EAX = _____, CF = _____, ZF = _____
(2) mov eax, 100 ; EAX = _____
    add eax, 200 ; EAX = _____, CF = _____
(3) mov eax, 100 ; EAX = _____
    add al, 200 ; EAX = _____, CF = _____
(4) mov al, 7fh ; AL = _____
    sub al, 8 ; AL = _____, CF = _____, SF = _____
    sub al, 80h ; AL = _____, CF = _____, OF = _____
    add al, 3 ; AL = _____, CF = _____, ZF = _____
```

- 3.12 给出下列各条指令执行后 AL 的值, 以及 CF、ZF、SF、OF 和 PF 的状态。

```
mov al, 80h
add al, al
add al, 80h
cmp al, 0f0ah
mov al, al
dec al
inc al
```

- 3.13 如下两段程序执行后, EDX、EAX 寄存器的值是多少?

```
(1) 加法程序
mov ebx, 11h
mov eax, 080000000h
add eax, 040000000h
adc ebx, 0
```

(2) 减法器

```
mov ebx, 100h
mov eax, 64000000h
sub eax, 84000000h
sbb ebx, 0
```

- 3.14 请分别用一条汇编语言指令完成如下功能。

- (1) 把 EBX 寄存器和 EDX 寄存器的内容相加, 并把结果存入 EDX 寄存器。
- (2) 用寄存器 EBX 和 ESI 的基址寻址方式把存储器中的一个字节与 AL 寄存器的内容相加, 并把结果送到 AL 中。
- (3) 用 EBX 和位位移量 0B2H 的寄存器相对寻址方式把存储器中的一个双字和 ECX 寄存器的内容相加, 并把结果送回寄存器中。
- (4) 将 32 位变量 VARD 与数 3412H 相加, 并把结果送回该存储单元中。
- (5) 把数 0A0H 与 EAX 寄存器的内容相加, 并把结果送回 EAX 中。

- 3.15 两个 64 位无符号整数分别存放在变量 buffer1 和 buffer2 中, 定义数据并编写代码完成 EDX: EAX=buffer1+buffer2 功能。

- 3.16 分别执行如下程序片段, 说明每条指令的执行结果:

```
(1) mov esi, 10011100b ; ESI = _____
    and esi, 80h ; ESI = _____
    or esi, 7fh ; ESI = _____
    xor esi, 0feh ; ESI = _____
(2) mov eax, 1010b ; EAX = B
    shr eax, 2 ; EAX = B, CF = _____
    shl eax, 1 ; EAX = B, CF = _____
    and eax, 3 ; EAX = B, CF = _____
(3) mov eax, 1011b ; EAX = B
    rol eax, 2 ; EAX = B, CF = _____
    ror eax, 1 ; EAX = B, CF = _____
    or eax, 3 ; EAX = B, CF = _____
(4) xor eax, eax ; EAX = _____, CF = _____
    ; ZF = _____, SF = _____, PF = _____
```

- 3.17 给出下列各条指令执行后 AX 的结果, 以及状态标志 CF、OF、SF、ZF、PF 的状态。

```
mov ax, 1470h
and ax, ax
or ax, ax
xor ax, ax
not ax
test ax, 0f0f0h
```

- 3.18 举例说明逻辑运算指令如何实现复位、置位和求反功能。
- 3.19 编写一个压缩 BCD 码变量 (如 92H) 转换为对应的 ASCII 码, 然后调用 DISPC 子程序 (在输入输出子程序库中) 显示。

- 3.20 有 4 个 32 位寄存器, 分别保存在 VAR1、VAR2、VAR3 和 VAR4 变量中, 阅读如下程序片段, 得出运算公式, 并说明运算结果存于何处。

```
mov eax, [var1]
```

```
imul [var2]
mov ebx, [var3] ; 实现 ebx 符号扩展到 ecx
mov ecx, ebx
mov ecx, 3
add ecx, ebx
adc ebx, ecx
sub ebx, 10
sbb ebx, 0
jnb [var1]
```

- 3.21 如下程序片段实现 EAX 乘以个数 X 的功能, 请判断 X 等于多少。
请使用一条乘法指令实现上述功能。

```
mov ecx, eax
shl ecx, 2
lea ecx, [ecx + ecx * 8]
sub ecx, ecx
```

- 3.22 请使用移位和加減指令编写一个程序片段计算 $EAX \times 21$, 假设乘积不超过 32 位。提示: $21 = 2^4 + 2^2 + 2^0$ 。
- 3.23 阅读如下程序, 为每条指令添加注释, 指出其功能或作用, 并说明这个程序运行后显示的结果。如果将程序中的寄存器间接寻址替换为寄存器相对寻址, 如何修改程序?

```
; 数据段
num db 6, 7, 8, 3, 0, 0, 0
count equ 6 - num
tab db "67783000"
; 代码段
mov ecx, count
mov esi, offset num
mov edi, offset tab
again: mov al, [esi]
        xchg al, [edi]
        mov [esi], al
        call dispac ; 显示 AL 中的字符
        add esi, 1
        add edi, 1
        loop again ; 循环结束
```

- 3.24 说明如下程序执行后的显示结果:

```
; 数据段
msg db "HELLOX000", 0
count equ 6 - msg
; 代码段
mov ecx, count - 1
mov ebx, offset msg
again: mov al, [ebx]
        add al, 20h
        mov [ebx], al
        add ebx, 1
        loop again
        mov ecx, offset msg
        call dispmsg
```

如果将其中的语句 “mov ebx, offset msg” 改为 “mov ebx, ebx”, 而利用 EDI 间接寻址的两个语句如何修改成 EDX 寄存器相对寻址, 就可以实现同样功能?

- 3.25 下面程序的功能是将数组 ARRAY1 的每个元素加固定值 (8000H), 并将保存在数组 ARRAY2 中。在空白处填入适当的语句或语句的一部分。

```
; 数据段
array1 dd 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
count equ ($ - array1) / 4
array2 db count dup (0)
; 代码段
mov ecx, count - 1
mov ebx, 0
again: mov ebx, array1[ebx * 4]
        add eax, 8000h
        add ebx, _____
        loop again
```

第4章习题

4.1 简答题

- (1) CPUID 指令返回识别字符串的首个字符“G”在哪个寄存器中?
- (2) 数据的直接寻址和指令的直接寻址有什么区别?
- (3) 什么特点决定了目标地址的相对寻址方式应用最多?
- (4) JCC 指令能跳转到代码段之外吗?
- (5) 什么是奇偶校验?
- (6) 标志位 JZ 和 JE 为什么表达同一条指令?
- (7) 为什么判断无符号数大小和有符号数大小的条件转移指令不同?
- (8) 双分支结构中两个分支体之间的 JMP 指令有什么作用?
- (9) 如果循环体的代码量远超过 128 个字节,还能用 LOOP 指令实现计数控制循环吗?
- (10) 什么是“先循环后判断”循环结构?

4.2 判断题

- (1) 单字节指令指针改变或者指令指针和代码段寄存器的改变都将引起程序流程的跳转。
- (2) 指令的相对寻址都是近转移。
- (3) 采用指令的寄存器间接寻址,目标地址来自存储单元。
- (4) JMP 指令对应高级语言的 GOTO 语句,所以不能使用。
- (5) 因为条件转移指令 JCC 要用标志作为条件,所以也影响标志。
- (6) JA 和 JG 指令的条件都是“大于”,所以是同一个指令的两个标志位。
- (7) JC 和 JB 的条件都是 CF=1,所以是同一指令。
- (8) 控制循环是否结束只能在一次循环结束之后进行。
- (9) 介绍 LOOP 指令时,常说它相当于 DEC ECX 和 JNZ 两条指令。但考虑对状态标志的影响,它们有差别。LOOP 指令不影响标志,而 DEC 指令会影响除 CF 之外的其他状态标志。
- (10) 若 ECX=0,则 LOOP 指令和 JECX 指令都发生转移。

4.3 填空题

- (1) JMP 指令根据目标地址的转移范围和寻址方式,可以分成 4 种类型:段内转移、_____,段内转移、_____,段间转移、_____,段间转移、_____。
- (2) MASM 给近转移、近转移和远转移定义的类型名依次是 _____、_____和 _____。
- (3) 假设在平展存储模型下,EBX=1256H,双字变量 TABLE 的偏移地址是 20A1H,线性地址 32F7H 处存放 3280H,执行指令“JMP EBX”后,EIP = _____,执行指令

第4章 程序结构 181

“JMP TABLE [EBX]”后,EIP = _____。

- (4) “CMP EAX, 3721H”指令之后是 JZ 指令,发生转移的条件是 EAX = _____,此时 ZF = _____。
- (5) 执行“SHR EBX, 4”指令后,JNC 发生转移,说明 EBX 的 D₄ = _____。
- (6) 在 EDX 等于 0 时转移,可以使用指令“CMP EDX, _____”,也可以使用指令“TEST EDX, _____”构成条件,然后使用 JE 指令实现转移。
- (7) 循环结构程序一般由三个部分组成,它们是 _____、循环体和 _____部分。
- (8) JECXZ 指令发生转移的条件是 _____,LOOP 指令不发生转移的条件是 _____。
- (9) LOOP 指令进行减 1 计数,实际应用中常进行加 1 计数。针对例 4-14 程序,如果删除其中的 LOOP 指令,则可以使用指令“CMP _____, ECX”和“JB _____”替代。
- (10) 小写字母“e”是英文当中出现频率最高的字母。如果某个英文文档利用例 4-16 的异或方法进行简单加密,统计发现密文中华字节数据“8FH”最多,则该程序采用的字节密码可能是 _____。

- 4.4 已知 var1、var2、var3 和 var4 是 32 位无符号整数(假设 var2 大于 7),用汇编语言程序片段实现如下 C++ 语句:

```
var4 = (var1 * 6) / (var2 - 7) + var3
```

- 4.5 已知 var1、var2、var3 和 var4 是 32 位有符号整数,用汇编语言程序片段实现如下 C++ 语句:

```
var1 = (var2 * var3) / (var4 + 8) - 47
```

- 4.6 参看例 4-1,假设 N 小于 90000,这时求和结果只需要 EAX 保存,EDX 为 0。修改例 4-1 使其可以从键盘输入一个数值 N(用 READUID 子程序),最后显示累加和(用 DISPUD 子程序)。

- 4.7 定义 COUNT(假设为 10)个元素的 32 位数组,输入元素编号(0~COUNT-1),利用 DISPHD 子程序输出其地址,利用 DISPDISD 子程序输出其值。

- 4.8 为了验证例 4-4 程序的执行路径,可以在每个标号前增加显示一个数字的功能(利用 DISPC 子程序),使得程序运行后显示数码 1234。

- 4.9 执行如下程序片段后,CMP 指令分别使得 5 个状态标志 CF、ZF、SF、OF 和 PF 为 0 还是为 1? 它使得哪些条件转移指令 JCC 的条件成立、发生转移?

```
mov eax,20h  
cmp eax,80h
```

- 4.10 判断下列程序段跳转的条件。

- (1) xor ax,1eleh
je equal
- (2) test al,10000001b
jnz here
- (3) cmp cx,64h
jb there

- 4.11 假设 EBX 和 ESI 存放的是有符号数,EDX 和 EDI 存放的是无符号数,请用比较指令和条件转移指令实现以下判断:

- (1) 若 $EDX > EDI$, 转到 `above` 执行。
 - (2) 若 $EBX > ESI$, 转到 `greater` 执行。
 - (3) 若 $EBX = 0$, 转到 `zero` 执行。
 - (4) 若 $EBX - ESI$ 产生溢出, 转到 `overflow` 执行。
 - (5) 若 $ESI \leq EBX$, 转到 `less_eq` 执行。
 - (6) 若 $EDI \leq EDX$, 转到 `below_eq` 执行。
- 4.12 使用“`SHR EAX, 2`”将 `EAX` 中的 D_1 位移入 `CF` 标志, 然后用 `JC/JNC` 指令替代 `JZ/JNZ` 指令完成例 4-6 程序的功能。
- 4.13 将例 4-7 程序修改为实现偶校验, 并进一步增加显示有关提示信息的功能, 使程序具有更加良好的交互性。
- 4.14 在采用奇偶校验传输数据的接收端应该验证数据传输的正确性。例如, 如果采用偶校验, 那么在接收到的数据中, 其包含“1”的个数应该为 0 或偶数个, 否则说明出现传输错误。现在, 在接收端编写一个这样的程序, 如果偶校验不正确显示错误信息, 传输正确则继续。假设传送字节数据、最高位作为校验位, 接收到的数据已经保存在 `Rdata` 变量中。
- 4.15 IA-32 处理器的指令 `CDQ` 将 `EAX` 符号扩展到 `EDX`。假设没有该指令, 编程实现该指令功能。
- (1) 按照符号扩展的含义编程, 即 EAX 最高位为 0, 则 $EDX = 0$; EAX 最高位为 1, 则 $EDX = \text{FFFFFFFFH}$ 。
 - (2) 使用移位等指令进行优化编程。
- 4.16 编写一个程序, 首先测试双字变量 `DVAR` 的最高位, 如果为 1, 则显示字母“L”, 如果最高位不为 1, 则继续测试最低位, 如果最低位为 1, 则显示字母“R”, 如果最低位也不为 1, 则显示字母“M”。
- 4.17 编写一个程序, 先提示输入数字“Input Number: 0~9”, 然后在下一行显示输入的数字, 结束; 如果不是输入了数字 0~9, 就提示错误“Error!”, 继续等待输入数字。
- 4.18 有一个首地址为 `ARRAY` 的 20 个双字的数组, 说明下列程序段的功能。

```

mov ecx, 20
mov eax, 0
mov esi, eax
sumlp: add eax, array[esi]
        add esi, 4
        loop sumlp
mov [total], eax

```

- 4.19 说明如下程序段的功能:

```

mov ecx, 16
mov bx, ax
next: shr ax, 1
        rcr edx, 1
        shr bx, 1
        rcr edx, 1
        loop next
mov eax, edx

```

- 4.20 编程将一个 64 位数据逻辑左移 3 位, 假设这个数据已经保存在 `EDX`。 `EAX` 寄存器对中。
- 4.21 编程中经常要记录某个字符出现的次数。现编程记录某个字符串中空格出现的次数, 将结果保存在 `SPACE` 单元。

- 4.22 将一个已经按升序排列的数组（第一个元素最小，后面逐渐增大）改为按降序排列（即第一个元素最大，后面逐渐减小）。实际上只要第一个元素与最后一个元素交换，第二个元素与倒数第二个元素交换，以此类推就可以，编程实现该功能。
- 4.23 编写计算 100 个 16 位正整数之和的程序。如果和不超过 16 位字的范围 (65 535)，则保存其和到 WORDSUM，如超过则显示 'Overflow!'。
- 4.24 在一个已知长度的字符串中查找是否包含“BUG”子字符串。如果存在，则显示“Y”，否则显示“N”。
- 4.25 主存中有一个 8 位压缩 BCD 码数据，保存在一个双字变量中。现在需要进行显示，但要求不显示前导 0。由于位数较多，需要利用循环实现，但如何处理前导 0 和数据中间的 0 呢？不妨设置一个标记。编程实现。
- 4.26 已知一个字符串的长度，删除其中所有的空格字符。请从字符串最后一个字符开始逐个向前判断并进行处理。
- 4.27 第 2 章习题 2.14 在屏幕上显示 ASCII 表，现仅在数据段设置表格缓冲区，编程将 ASCII 代码值填入留出位置的表格，然后调用显示功能实现（需要利用双重循环）。
- 4.28 快速乘法程序。只使用移位和加减法指令将两个任意的 32 位无符号整数相乘，求出乘积（假设不超过 32 位）。使用这个程序显示 3567×7653 的结果。算法的基本思想是将较小数进行右移决定是否累加，对较大数左移位并进行累加。
- 4.29 素数判断程序。编写一个程序，提示用户输入一个数字，然后显示信息说明该数字是否是素数。素数 (Prime) 是指只能被自身和 1 整除的自然数。
- (1) 采用直接简单的算法：假设输入 N ，将其逐个除以 $2 \sim N-1$ ，只要能整除（余数为 0）说明不是素数，只有都不能整除才是素数。
 - (2) 采用只对奇数整除的算法：1、2 和 3 是素数，所有大于 3 的偶数不是素数，从 5 开始的数字只需除以从 3 开始的奇数，只有都不能整除才是素数。
- 4.30 计算素数个数程序。使用 Eratosthenes 筛法，求 $1 \sim 100\,000$ 之间共有多少个素数。
- Eratosthenes 筛法是希腊数学家 Eratosthenes 发明的算法，给出了一种找出给定范围内所有素数的快速方法。该算法要求创建一个字节数组，以如下方式将不是素数的位置标记出来：2 是一个素数，从位置 2 开始，把所有 2 的倍数的位置标记 1；2 之后的素数是 3，将 3 的倍数的位置标记 1；3 之后的素数是 5，将 5 的倍数的位置标记 1；如此重复，直到标记所有非素数的位置。处理结束，数组中没有被标记的位置都对应一个素数。
- (1) 如果将该数组事先定义在数据段，则生成的可执行文件中将包括这个数组，形成很大的一个文件。MASM 提供了一个定义无初始化数据段的伪指令 (.DATA?)，在其下定义的变量不能有初值，因为它们在程序执行时才被分配存储空间。但可以利用一段循环程序将初值 0 全部填入其中。伪指令“.DATA?”对定义大量的无初值的变量（如数组）特别有效。建议本程序采用这个方法。
 - (2) 即使是在程序执行时才分配存储空间，对于求更大范围内的素数，存储空间仍然是一个编程关键。可以考虑进一步用一个二进制位表示一个数字的方法，这样一个字节就可以表达 8 个数字，存储空间可以减少为原来的八分之一。

第5章习题

5.1 简答题

- (1) 指令“CALL EBX”采用了什么寻址方式?
- (2) 为什么 MASM 要求使用 PROC 定义子程序?
- (3) 为什么特别强调为子程序加上必要的注释?
- (4) 参数传递的“传值”和“传址”有什么区别?
- (5) 子程序采用堆栈传递参数,为什么要特别注意堆栈平衡问题?
- (6) INCLUDE 语句和 INCLUDELIB 语句有什么区别?
- (7) 什么是子程序库?
- (8) 调用宏时没有为形参提供实参会怎样?
- (9) 宏定义体中的标号为什么要用 LOCAL 伪指令声明?
- (10) 条件汇编不成立的语句会出现在可执行文件中吗?

5.2 判断题

- (1) 过程定义 PROC 是一条处理器指令。
- (2) CALL 指令的执行并不影响堆栈指针 ESP。
- (3) CALL 指令本身不能包含子程序的参数。

- (4) CALL 指令用在调用程序中,如果被调用程序中也有 CALL 指令,说明出现了嵌套。
- (5) 子程序需要保护寄存器,包括保护传递入口参数和出口参数的通用寄存器。
- (6) 利用 INCLUDE 包含的源文件实际上只是源程序的一部分。
- (7) 宏调用与子程序调用一样都要使用 CALL 指令实现。
- (8) 宏定义与子程序一样,一般书写在主程序之后。
- (9) 重复汇编类似于宏汇编,需要先定义后调用。
- (10) 条件汇编并不像条件转移指令那样使用标志作为条件。

5.3 填空题

- (1) 指令“RET il6”的功能相当于“RET”指令和“ADD ESP, _____”的组合。
- (2) 对于例 5-1 程序中的 RET 指令,如果用 POP EBP 指令和 JMP EBP 指令替换,则 EBP 内容是_____。
- (3) 子程序的参数传递主要有 3 种,它们是_____、_____和_____。
- (4) 数值 10 在计算机内部用二进制“1010”编码表示,用十六进制表还是_____。如果将该编码加 37H,则为_____,它是字符_____的 ASCII 码值。
- (5) 利用堆栈传递子程序参数的方法是固定的,例如,寻址堆栈段数据的寄存器是_____。
- (6) MASM 汇编语言中,声明一个共用的变量应使用_____伪指令;而使用外部变量要使用_____伪指令声明。
- (7) 过程定义开始是“TEST PROC”语句,则过程定义结束的语句是_____。宏定义开始是“DISP MACRO”语句,则宏定义结束的语句是_____。
- (8) 一个宏定义开始语句是“WriteChar MACRO CHAR; REQ”,则宏名是_____,参数有_____个,并且使用“;REQ”说明该参数_____。
- (9) 实现“DB 20 DUP (20H)”语句的功能也可以使用重复汇编,第 1 个语句是_____,第 2 个语句是“DB 20H”,第 3 个语句是_____。
- (10) 条件汇编语句“IF NUM LT 100”中的 LT 表示_____,该语句需要配合_____语句结束条件汇编。

5.4 如下子程序完成对 ECX 个元素的数组(由 EBX 指向其首地址)的求和,通过 EDX 和 EAX 返回结果,但程序有错误,请改正。

```
crazy PROC
    push eax
    xor eax, eax
    xor edx, edx
again: add eax, [ebx]
    adc edx, 0
    add ebx, 4
    loop again
    ret
ENDP crazy
```

5.5 请按如下说明编写子程序。

- 子程序功能:把用 ASCII 码表示的两位十进制数转换为压缩 BCD 码。
- 入口参数:DH=十位数的 ASCII 码,DL=个位数的 ASCII 码。
- 出口参数:AL=对应 BCD 码。

5.6 乘法的非压缩 BCD 码调整指令 AAM 执行的操作是:AH←AL÷10 的商,AL←AL÷10 的余数。利用 AAM 可以实现将 AL 中的 100 内数据转换为 ASCII 码,程序如下:

```
xor ah,ah
aam
add ax,3030h
```

利用这段程序,编写一个显示 AL 中数值(0~99)的子程序。

- 5.7 编写一个源程序,在键盘上按一个键,将其返回的 ASCII 码值显示出来,如果按下退格键(对应的 ASCII 码是 08H)则程序退出。请调用书中的 HTOASC 子程序。
- 5.8 编写一个子程序,它以二进制形式显示 EAX 中 32 位数据,并设计一个主程序验证。
- 5.9 将例 5-4 中的 32 位寄存器改用 16 位寄存器,仅实现输出 $-2^{15} \sim +2^{15}-1$ 之间的数据。
- 5.10 参考例 5-6,编写实现 32 位无符号整数输入的子程序,并设计一个主程序进行验证。
- 5.11 编写一个计算字节校验和的子程序。所谓“校验和”是指不计进位的累加,常用于检查信息的正确性。主程序提供入口参数,包括数据个数和数据缓冲区的首地址。子程序回送求和结果这个出口参数。
- 5.12 编制 3 个子程序,把一个 32 位二进制数用 8 位十六进制形式在屏幕上显示出来,分别运用如下 3 种参数传递方法,并配合 3 个主程序验证它。
 - (1) 采用 EAX 寄存器传递这个 32 位二进制数。
 - (2) 采用 temp 变量传递这个 32 位二进制数。
 - (3) 采用堆栈方法传递这个 32 位二进制数。
- 5.13 利用十六进制字节显示子程序 DISPHB 设计一个从低地址到高地址逐字节显示某个主存区域内容的子程序 DISPMEM。其入口参数为: EAX = 主存偏移地址, ECX = 字节个数(主存区域的长度)。同时编写一个主程序进行验证。
- 5.14 数据输入/输出程序。使用有符号十进制数据输入(例 5-6)、求平均值(例 5-7)以及显示子程序(例 5-4),编程实现从键盘输入 10 个数据,并输出它们的平均值。
 - (1) 编写主程序文件:定义必要的变量和交互信息,调用子程序输入 10 个数据,求平均值,然后输出。
 - (2) 编写子程序文件:包括 3 个子程序的过程定义。
 - (3) 说明进行模块连接的开发过程并上机实现。
 - (4) 将子程序文件形成一个子程序库,说明开发过程并上机实现。
- 5.15 区别如下概念:宏定义、宏调用、宏指令、宏展开、宏汇编。
- 5.16 如何定义和调用宏?
- 5.17 宏结构和子程序在应用中有什么不同?如何选择采用何种结构?
- 5.18 编写一个宏 SWAP,参数是两个 32 位寄存器或存储器操作数,宏定义体实现两个操作数的位置交换,包括两个都是存储器操作数的情况。
- 5.19 定义一个使用逻辑指令的宏 LOGICAL。
 - (1) 用它代表 4 条逻辑运算指令: AND、OR、XOR、TEST。可以使用 3 个形式参数,并给出一个宏调用以及对应宏展开的例子。
 - (2) 必要时做一点修改,使该宏能够包括 NOT 指令,给出一个使用 NOT 指令的宏调用以及对应宏展开的例子。
- 5.20 有一个宏定义:

```
defstr macro name,num,string
name&num db 'string',0
endm
```

给出如下宏调用的宏展开:

156 32 位汇编语言程序设计 第 3 版

- (1) defstr msg,4,<Chapter 4:Program Structure>
- (2) defstr msg,5,<Chapter 5:Procedure Programming>
- 5.21 定义一个宏“movestr strN, dst, sstr”,它将 strN 个字符从一个字符区 sstr 传送到另一个字符区 dst。

假设数据段定义如下缓冲区,请使用上述宏的调用实现 STRING1 到 STRING2 的传送。

```
string1 db 'In a major matter, no details are small.',0
count equ $-string1
string2 db count dup(0)
```
- 5.22 利用重复汇编方法定义一个数据区,数据区有 100 个双字,每个双字的高字部分依次是 2,4,6,...,200,低字部分都是 0。
- 5.23 利用宏结构完成以下功能:如果名为 COUNT 的数据大于 5,指令“ADD EAX, EAX”将汇编 10 次,否则什么也不汇编。
- 5.24 用宏结构实现宏指令 FINSUM,它比较两个数 varx 和 vary,若 $\text{varx} \geq \text{vary}$,则执行 $\text{sum} = \text{varx} + 8 \times \text{vary}$,否则执行 $\text{sum} = 4 \times \text{varx} + \text{vary}$ 。

